

PE2-23EE1613 : Data Analytics Lab

PE2-23EE1613 : Data Analytics Lab

Yeshwantrao Chavan College of Engineering, Nagpur

Department of Electronics Engineering

PE2-23EE1613 : Data Analytics Lab

6th Semester

Course Teacher : Dr. Shweta Dhondse

Introduction to Python

Python is a widely used programming language which allows programmers to write the program in fewer lines of code. It was designed by *Guido van Rossum* in 1991. Python uses an interpreter instead of compiler. Basically, the main difference between compiler and interpreter is that the compiler takes the code as 'whole' and compiles it. For example, in C language and on the other side in interpreter. For example: languages like Python. It interprets the written code 'line by line', and if somewhere error is found. It stops interpreting the code from there itself.

Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc). Python has a simple syntax similar to the English language. Python has syntax that allows developers to write programs with fewer lines than some other programming languages. Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.

Python can be treated in a procedural way, an object-orientated way or a functional way.

Features of Python

Open source: Python is publicly available open source software, any one can use source code that doesn't cost anything.

Easy-to-learn: Popular (scripting/extension) language, clear and easy syntax, no type declarations, automatic memory management, high-level data types and operations, design to read (more English like syntax) and write (shorter code compared to C, C++, and Java) fast.

High-level Language: High-level language (closer to human) refers to the higher level of concept from machine language (for example assembly languages). Python is an example of a high-level language like C, C++, Perl, and Java with low-level optimization.

Portable:

High level languages are portable, which means they are able to run across all major hardware and software platforms with few or no change in source code. Python is portable and can be used on Linux, Windows, Macintosh, Solaris, FreeBSD, OS/2, Amiga, AROS, AS/400 and many more.

Object-Oriented: Python is a full-featured object-oriented programming language, with features such as classes, inheritance, objects, and overloading.

Python is Interactive:

Python has an interactive console where you get a Python prompt (command line) and interact with the interpreter directly to write and test your programs. This is useful for mathematical programming.

Interpreted: Python programs are interpreted, takes source code as input, and then compiles (to portable byte-code) each statement and executes it immediately. No need to compiling or linking

Extendable : Python is often referred to as a "glue" language, meaning that it is capable to work in mixed-language environment. The Python interpreter is easily extended and can add a new built-in function or modules written in C/C++/Java code.

Libraries : Databases, web services, networking, numerical packages, graphical user interfaces, 3D graphics, others.

Supports: Support from online Python community

What is IDE:

- Integrated Development Environment (IDE): IDEs are full-fledged environment which provide all essential tools needed for software development. It just doesn't handle code (for example, write, edit, syntax highlighting and auto-completion) but also provides other features such as debugging, execution, testing, and code formatting that helps programmers.

Different IDE's for Python:

As per Level

- Beginner — IDLE (or Online Python Editors) is perfect choice for first steps in python language. PyCharm is also good but takes the help of some experienced person while using this.
- Intermediate — PyCharm, Sublime, Atom, Vs Code.
- Advanced — PyCharm, Vim, Emacs, Sublime, Atom, Vs Code.

As per Goal

- Web development — PyCharm Professional, VS Code
- Data Science — Spyder, Jupyter Notebook, PyCharm Professional
- Scripting — Sublime, Atom, PyCharm Community, Eclipse + PyDev
- QA — Sublime, Atom, PyCharm Community, Jupyter Notebook

PE2-23EE1613 : Data Analytics Lab

As per Environment/OS

- Linux, macOS — PyCharm, Sublime, Atom, Vim, Jupyter
- Windows — Sublime, VS Code, Eclipse + PyDev, PyCharm
- Multiple/mixed OS — PyCharm, Sublime, Atom
-

Features of Jupiter Notebook:

- Jupyter Notebook is a web-based interactive development environment;
 - It's well known in data science community for analyzing, sharing and presenting information.
 - It is easy to use, open-source software that allows you to create and share live code, visualizations, etc.
 - Some of its good features are given below...
1. Support for Numerical simulation, data cleaning, machine learning, data visualization, and statistical modeling.
 2. Markdown and HTML integration.
 3. Integrated data science libraries (matplotlib, NumPy, Pandas).
 4. It offers you to see and edit your code to create powerful presentations.
 5. Your complete work can be converted to PDF and HTML files, or you can just export it as a .py file.
- Starting and stopping servers, opening folders and files

Installing Anaconda

This article is all about installing and troubleshooting Anaconda and Spyder on different platforms.

Windows

Step 1 -> Download the installer from here

Download Link. Make sure that you download the version compatible to your operating system architecture. If you have a 64 bit OS then you can download either 64 bit or 32 bit, but if you have 32 bit OS then you should download the 32 bit version only.

Step 2 -> Double click on the .exe file that you downloaded just now and follow the instructions. If you are not sure of the installation options then just choose the default.

Step 3 -> To test your installation, open cmd (go to Start -> then type cmd in the search bar and hit enter). There type **conda list**. You will see a list of packages installed if not, then there was some problem.

Variables

Creating Variables

- Variables are containers for storing data values.
- Unlike other programming languages, Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.

Variable Names

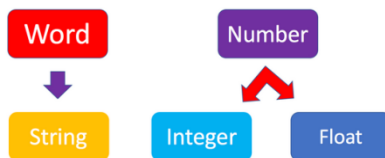
- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- Variable name must start with a letter or the underscore character
- Variable name cannot start with a number
- Variable name can only contain alpha-numeric characters & underscores (A-z, 0-9, & _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Types of objects in Python

- Python is an object-oriented language. There are many different types of objects in Python.
- The most common object types: *strings*, *integers* and *floats*.
- If you write words (text) in Python, you're using *character strings* (strings for short).
- The most common numbers, are *integers* (e.g. -1, 0, 100) and
- *floats*, which represent real numbers (e.g. 3.14, -42.0).



Or

Data Types in Python

Built-in Data Types

Data type is an important concept in programming

Variables can store data of different types, and different types can do different things.

Python has following data types built-in by default, in these categories:

- Text Type: str
- Numeric Types: int, float, complex
- Sequence Types: list, tuple, range
- Mapping Type: dict
- Set Types: set, frozenset
- Boolean Type: bool
- Binary Types: bytes, bytearray, memoryview

Getting the Data Type

To get data type of any object by using type() function:

Example

Print data type of variable x:

```
x = 7
```

```
print(type(x)) (Try it)          #<class 'int'>
```

Python standard library

The Python Standard Library contains the exact syntax, semantics, and tokens of Python. It contains built-in modules that provide access to basic system functionality like I/O and some other core modules. Most of the Python Libraries are written in the C programming language. The Python standard library consists of more than 200 core modules. All these work together to make Python a high-level programming language. Python Standard Library plays a very important role. Without it, the programmers can't have access to the functionalities of Python. But other than this, there are several other libraries in Python that make a programmer's life easier. Some of the commonly used libraries are:

1. **TensorFlow:** This library was developed by Google in collaboration with the Brain Team. It is an open-source library used for high-level computations. It is also used in machine learning and deep learning algorithms. It contains a large number of tensor operations. Researchers also use this Python library to solve complex computations in Mathematics and Physics.
2. **Matplotlib:** This library is responsible for plotting numerical data. And that's why it is used in data analysis. It is also an open-source library and plots high-defined figures like pie charts, histograms, scatterplots, graphs, etc.
3. **Pandas:** Pandas are an important library for data scientists. It is an open-source machine learning library that provides flexible high-level data structures and a variety of analysis tools. It eases data analysis, data manipulation, and cleaning of data. Pandas support operations like Sorting, Re-indexing, Iteration, Concatenation, Conversion of data, Visualizations, Aggregations, etc.
4. **Numpy:** The name "Numpy" stands for "Numerical Python". It is the commonly used library. It is a popular machine learning library that supports large matrices and multi-dimensional data. It consists of in-built mathematical functions for easy computations. Even libraries like TensorFlow use Numpy internally to perform several operations on tensors. Array Interface is one of the key features of this library.
5. **SciPy:** The name "SciPy" stands for "Scientific Python". It is an open-source library used for high-level scientific computations. This library is built over an extension of Numpy. It works with Numpy to handle complex computations. While Numpy allows sorting and indexing of array data, the numerical data code is stored in SciPy. It is also widely used by application developers and engineers.
6. **Scrapy:** It is an open-source library that is used for extracting data from websites. It provides very fast web crawling and high-level screen scraping. It can also be used for data mining and automated testing of data.

7. **Scikit-learn:** It is a famous Python library to work with complex data. Scikit-learn is an open-source library that supports machine learning. It supports variously supervised and unsupervised algorithms like linear regression, classification, clustering, etc. This library works in association with Numpy and SciPy.
8. **PyGame:** This library provides an easy interface to the Standard Directmedia Library (SDL) platform-independent graphics, audio, and input libraries. It is used for developing video games using computer graphics and audio libraries along with Python programming language.
9. **PyTorch:** PyTorch is the largest machine learning library that optimizes tensor computations. It has rich APIs to perform tensor computations with strong GPU acceleration. It also helps to solve application issues related to neural networks.
10. **PyBrain:** The name “PyBrain” stands for Python Based Reinforcement Learning, Artificial Intelligence, and Neural Networks library. It is an open-source library built for beginners in the field of Machine Learning. It provides fast and easy-to-use algorithms for machine learning tasks. It is so flexible and easily understandable and that’s why is really helpful for developers that are new in research fields.

Use of Libraries in Python Program

```
# Importing math library
import math
```

```
A = 16
print(math.sqrt(A))
```

Importing specific items from a library module

```
# Importing specific items
from math import sqrt, sin
```

```
A = 16
B = 3.14
print(sqrt(A))
print(sin(B))
```

NumPy

NumPy, short for Numerical Python, is the foundational package for scientific computing in Python. The majority of this book will be based on NumPy and libraries built on top of NumPy. It provides, among other things:

- A fast and efficient multidimensional array object *ndarray*
- Functions for performing element-wise computations with arrays or mathematical operations between arrays
- Tools for reading and writing array-based data sets to disk
- Linear algebra operations, Fourier transform, and random number generation
- Tools for integrating connecting C, C++, and Fortran code to Python

Beyond the fast array-processing capabilities that NumPy adds to Python, one of its primary purposes with regards to data analysis is as the primary container for data to be passed between algorithms. For numerical data, NumPy arrays are a much more efficient way of storing and manipulating data than the other built-in Python data structures. Also, libraries written in a lower-level language, such as C or Fortran, can operate on the data stored in a NumPy array without copying any data.

Pandas

Pandas combines the high performance array-computing features of NumPy with the flexible data manipulation capabilities of spreadsheets and relational databases (such as SQL). It provides sophisticated indexing functionality to make it easy to reshape, slice and dice, perform aggregations, and select subsets of data. pandas is the primary tool that we will use in this book.

Matplotlib

matplotlib is the most popular Python library for producing plots and other 2D data visualizations. It was originally created by John D. Hunter (JDH) and is now maintained by a large team of developers. It is well-suited for creating plots suitable for publication. It integrates well with IPython (see below), thus providing a comfortable interactive environment for plotting and exploring data. The plots are also interactive; you can zoom in on a section of the plot and pan around the plot using the toolbar in the plot window.

IPython

IPython is the component in the standard scientific Python toolset that ties everything together. It provides a robust and productive environment for interactive and exploratory computing. It is an enhanced Python shell designed to accelerate the writing, testing, and debugging of Python code. It is particularly useful for interactively working with data and visualizing data with matplotlib.

SciPy

SciPy is a collection of packages addressing a number of different standard problem domains in scientific computing. Here is a sampling of the packages included:

- `scipy.integrate`: numerical integration routines and differential equation solvers
- `scipy.linalg`: linear algebra routines and matrix decompositions extending beyond those provided in `numpy.linalg`.
- `scipy.optimize`: function optimizers (minimizers) and root finding algorithms
- `scipy.signal`: signal processing tools
- `scipy.sparse`: sparse matrices and sparse linear system solvers
- `scipy.special`: wrapper around SPECFUN, a Fortran library implementing many common mathematical functions, such as the gamma function
- `scipy.stats`: standard continuous and discrete probability distributions (density)

PE2-23EE1613 : Data Analytics Lab

functions, samplers, continuous distribution functions), various statistical tests, and more descriptive statistics

- `scipy.weave`: tool for using inline C++ code to accelerate array computations

Together NumPy and SciPy form a reasonably complete computational replacement for much of MATLAB along with some of its add-on toolboxes.

Lab 1: Introduction to Python tool for data analytics

PE2-23EE1613 : Data Analytics Lab

NumPy, short for Numerical Python, is the fundamental package required for high performance scientific computing and data analysis. It is the foundation on which nearly all of the higher-level tools in this book are built. Here are some of the things it provides:

- ndarray, a fast and space-efficient multidimensional array providing vectorized arithmetic operations and sophisticated broadcasting capabilities
- Standard mathematical functions for fast operations on entire arrays of data without having to write loops
- Tools for reading / writing array data to disk and working with memory-mapped files
- Linear algebra, random number generation, and Fourier transform capabilities
- Tools for integrating code written in C, C++, and Fortran

Creating ndarrays

The easiest way to create an array is to use the array function. This accepts any sequence-like object (including other arrays) and produces a new NumPy array containing the passed data. For example, a list is a good candidate for conversion:

```
In [13]: data1 = [6, 7.5, 8, 0, 1]
In [14]: arr1 = np.array(data1)
In [15]: arr1
Out[15]: array([ 6. , 7.5, 8. , 0. , 1. ])
```

Nested sequences, like a list of equal-length lists, will be converted into a multidimensional array:

```
In [16]: data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
In [17]: arr2 = np.array(data2)
In [18]: arr2
Out[18]:
```

```
array([[1, 2, 3, 4], [5, 6, 7, 8]])
In [19]: arr2.ndim
Out[19]: 2
```

```
In [20]: arr2.shape
Out[20]: (2, 4)
```

Function	Description
array	Convert input data (list, tuple, array, or other sequence type) to an ndarray either by inferring a dtype or explicitly specifying a dtype. Copies the input data by default.
sarray	Convert input to ndarray, but do not copy if the input is already an ndarray
arange	Like the built-in range but returns an ndarray instead of a list.

PE2-23EE1613 : Data Analytics Lab

ones, ones_like Produce an array of all 1's with the given shape and dtype. ones_like takes another	Produce an array of all 1's with the given shape and dtype. ones_like takes another array and produces a ones array of the same shape and dtype
zeros, zeros_like Like ones and ones_like but producing	Like ones and ones_like but producing arrays of 0's instead
empty, empty_like	Create new arrays by allocating new memory, but do not populate with any values like ones and zeros
eye, identity	Create a square N x N identity matrix (1's on the diagonal and 0's elsewhere)

Step 1: Importing the libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Step 2: Import the dataset

Importing a data set:-

```
df = pd.read_csv('train.csv')
```

```
df.info()
```

```
<
```

If you carefully observe the above summary of Pandas, there are 891 total rows but Age shows only 714 (which means we're missing some data), Embarked is missing two rows and Cabin is missing a lot as well. Object data types are non-numeric so we have to find a way to encode them to numerical values.

Working with Numpy arrays

Ex1) Creating array object

```
arr = np.array( [[ 1, 2, 3], [ 4, 2, 5]] )
```

```
# Printing type of arr object
```

```
print("Array is of type: ", type(arr))
```

```
# Printing array dimensions (axes)
```

```
print("No. of dimensions: ", arr.ndim)

# Printing shape of array print("Shape of array: ", arr.shape)

# Printing size (total number of elements) of array

print("Size of array: ", arr.size)

# Printing type of elements in array

print("Array stores elements of type: ", arr.dtype)
```

Ex2) The easiest way to create an array is to use the array function. This accepts any sequence-like object (including other arrays) and produces a new NumPy array containing the passed data. For example, a list is a good candidate for conversion:

```
In [13]: data1 = [6, 7.5, 8, 0, 1]
```

```
In [14]: arr1 = np.array(data1)
```

```
In [15]: arr1
```

```
Out[15]: array([ 6. , 7.5, 8. , 0. , 1. ])
```

Nested sequences, like a list of equal-length lists, will be converted into a multidimensional array:

```
In [16]: data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
```

```
In [17]: arr2 = np.array(data2)
```

```
In [18]: arr2
```

```
Out[18]: array([[1, 2, 3, 4], [5, 6, 7, 8]])
```

```
In [19]: arr2.ndim
```

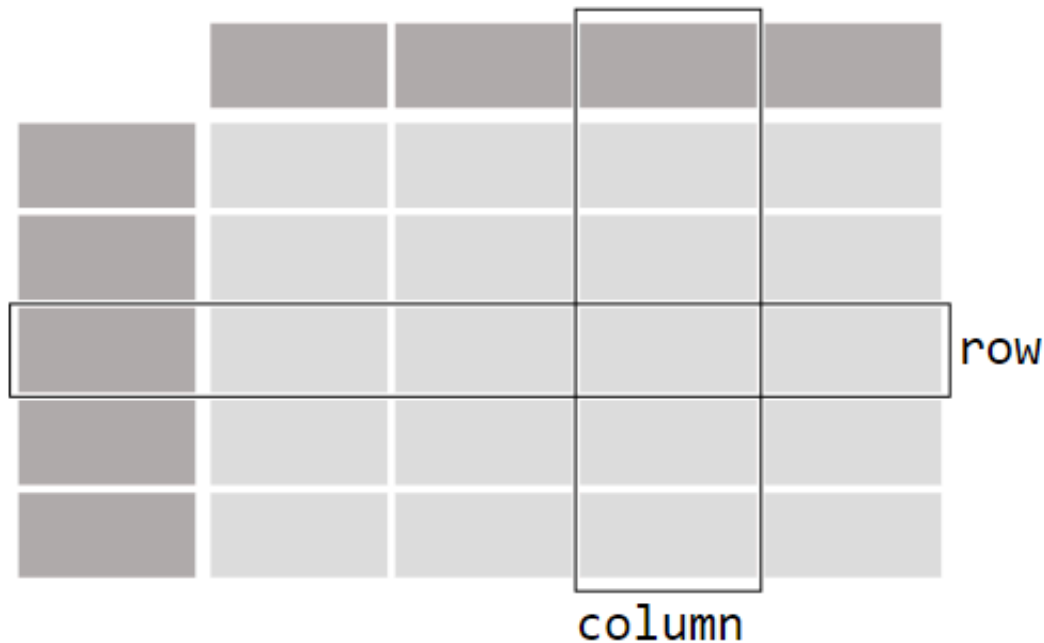
```
Out[19]: 2
```

```
In [20]: arr2.shape
```

```
Out[20]: (2, 4)
```

When working with tabular data, such as data stored in spreadsheets or databases, pandas is the right tool for you. pandas will help you to explore, clean, and process your data. In pandas, a data table is called a

DataFrame



Installation of Pandas

If you have Python and PIP already installed on a system, then installation of Pandas is very easy.

Install it using this command:

```
C:\Users\Your Name>pip install pandas
```

If this command fails, then use a python distribution that already has Pandas installed like, Anaconda, Spyder etc.

Import Pandas

Once Pandas is installed, import it in your applications by adding the import keyword:

```
import numpy as np
```

import pandas as pd

Object creation

Ex 3: Creating a Series by passing a list of values, letting pandas create a default integer index:

```
>>>
```

```
In [3]: s = pd.Series([1, 3, 5, np.nan, 6, 8])
```

```
In [4]: s
```

Out[4]:

```
0    1.0
1    3.0
2    5.0
3   NaN
4    6.0
5    8.0
```

```
dates = pd.date_range("20130101", periods=6)
```

```
In [6]: dates
```

Out[6]:

```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')
```

```
In [7]: df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
```

```
In [8]: df
```

Out[8]:

```
      A      B      C      D
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
2013-01-06 -0.673690  0.113648 -1.478427  0.524988
```

Creating a **DataFrame** by passing a dictionary of objects that can be converted into a series-like structure:

```
ex2: data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)

#refer to the row index:
print(df.loc[0])

#use a list of indexes:
print(df.loc[[0, 1]])
```

Use the named index in the loc attribute to return the specified row(s).

Return "day2":

```
#refer to the named index:
print(df.loc["day2"])
```

```
ex 4 #load data into a DataFrame object:
df = pd.DataFrame(data)
```

```
print(df)
```

```
ex: mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}
myvar = pd.DataFrame(mydataset)
```

```
print(myvar)
```

Example:

```
df2 = pd.DataFrame({ "A": 1.0,"B": pd.Timestamp("20130102"),"C": pd.Series(1,
index=list(range(4)), dtype="float32"),
    "D": np.array([3] * 4, dtype="int32"),
    "E": pd.Categorical(["test", "train", "test", "train"]),
```



```
"F": "foo",})
```

```
In [10]: df2
```

```
Out[10]:
```

```

  A      B  C D  E  F
0  1.0 2013-01-02  1.0 3  test foo
1  1.0 2013-01-02  1.0 3  train foo
2  1.0 2013-01-02  1.0 3  test foo
3  1.0 2013-01-02  1.0 3  train foo
```

The columns of the resulting DataFrame have different dtypes:

```
>>>
```

```
In [11]: df2.dtypes
```

```
Out[11]:
```

```

A      float64
B      datetime64[ns]
```

```

C      float32
D      int32
E      category
F      object
dtype: object
```

```
df.head()
```

```
Out[13]:
```

```

      A      B      C      D
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
```

```
In [14]: df.tail(3)
```

```
Out[14]:
```

```

      A      B      C      D
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
2013-01-06 -0.673690  0.113648 -1.478427  0.524988
```

DataFrame.to_numpy() gives a NumPy representation of the underlying data. Note that this can be an expensive operation when your **DataFrame** has columns with different data types, which comes down to a fundamental difference between pandas and NumPy: **NumPy arrays have one dtype for the entire array, while pandas DataFrames have one dtype per column.** When you call **DataFrame.to_numpy()**, pandas will find the NumPy dtype that can hold *all* of the dtypes in the DataFrame. This may end up being object, which requires casting every value to a Python object.

For df, our **DataFrame** of all floating-point values, and **DataFrame.to_numpy()** is fast and doesn't require copying data:

```
>>>
In [17]: df.to_numpy()
Out[17]:
array([[ 0.4691, -0.2829, -1.5091, -1.1356],
       [ 1.2121, -0.1732,  0.1192, -1.0442],
       [-0.8618, -2.1046, -0.4949,  1.0718],
       [ 0.7216, -0.7068, -1.0396,  0.2719],
       [-0.425 ,  0.567 ,  0.2762, -1.0874],
       [-0.6737,  0.1136, -1.4784,  0.525 ]])
```

For df2, the **DataFrame** with multiple dtypes, **DataFrame.to_numpy()** is relatively expensive:

```
>>>
In [18]: df2.to_numpy()
Out[18]:
array([[1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo']],
      dtype=object)
```

```
df.describe()
Out[19]:
```

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	0.073711	-0.431125	-0.687758	-0.233103
std	0.843157	0.922818	0.779887	0.973118
min	-0.861849	-2.104569	-1.509059	-1.135632
25%	-0.611510	-0.600794	-1.368714	-1.076610
50%	0.022070	-0.228039	-0.767252	-0.386188
75%	0.658444	0.041933	-0.034326	0.461706
max	1.212112	0.567020	0.276232	1.071804

Transposing your data:

```
>>>
```

```
In [20]: df.T
```

```
Out[20]:
```

```
2013-01-01  0.469112  1.212112 -0.861849  0.721555 -0.424972 -0.673690
A  -0.282863 -0.173215 -2.104569 -0.706771  0.567020  0.113648
B  -1.509059  0.119209 -0.494929 -1.039575  0.276232 -1.478427
C  -1.135632 -1.044236  1.071804  0.271860 -1.087401  0.524988
D
```

DataFrame.sort_index() sorts by an axis:

```
>>>
```

```
In [21]: df.sort_index(axis=1, ascending=False)
```

```
Out[21]:
```

```
          D      C      B      A
2013-01-01 -1.135632 -1.509059 -0.282863  0.469112
2013-01-02 -1.044236  0.119209 -0.173215  1.212112
2013-01-03  1.071804 -0.494929 -2.104569 -0.861849
2013-01-04  0.271860 -1.039575 -0.706771  0.721555
2013-01-05 -1.087401  0.276232  0.567020 -0.424972
2013-01-06  0.524988 -1.478427  0.113648 -0.673690
```

DataFrame.sort_values() sorts by values:

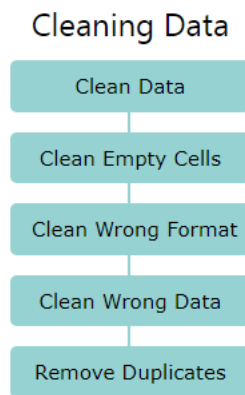
```
>>>
```

```
In [22]: df.sort_values(by="B")
```

```
Out[22]:
```

```
          A      B      C      D
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-06 -0.673690  0.113648 -1.478427  0.524988
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
```

Lab 2: Data Cleaning With pandas and NumPy



Data Cleaning: Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

Ex: 1 Return a new Data Frame with no empty cells:

```
import pandas as pd
df = pd.read_csv('data.csv')
new_df = df.dropna()
print(new_df.to_string())
```

By default, the dropna() method returns a *new* DataFrame, and will not change the original.

Ex: 2 Remove all rows with NULL values:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace = True)
print(df.to_string())
```

Now, the dropna(inplace = True) will NOT return a new DataFrame, but it will remove all rows containing NULL values from the original DataFrame.

Ex: 3 Replace Empty Values

Another way of dealing with empty cells is to insert a *new* value instead. This way you do not have to delete entire rows just because of some empty cells. The fillna() method allows us to replace empty cells with a value:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna(130, inplace = True)
```

Ex4: Replace Only For Specified Columns

The example above replaces all empty cells in the whole Data Frame. To only replace empty values for one column, specify the *column name* for the DataFrame:

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')  
df["Calories"].fillna(130, inplace = True)
```

Ex5: Replace Using Mean, Median, or Mode

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.

Pandas uses the mean() median() and mode() methods to calculate the respective values for a specified column:

```
import pandas as pd  
df = pd.read_csv('data.csv')  
x = df["Calories"].mean()  
df["Calories"].fillna(x, inplace = True)
```

Ex6: Calculate the MEDIAN, and replace any empty values with it:

```
import pandas as pd  
df = pd.read_csv('data.csv')  
x = df["Calories"].median()  
df["Calories"].fillna(x, inplace = True)
```

Median = the value in the middle, after you have sorted all values ascending.

```
import pandas as pd
```

```
df = pd.read_csv('data.csv')  
x = df["Calories"].mode()[0]  
df["Calories"].fillna(x, inplace = True)
```

Mode = the value that appears most frequently.

Data of Wrong Format

Cells with data of wrong format can make it difficult, or even impossible, to analyze data.

To fix it, you have two options: remove the rows, or convert all cells in the columns into the same format.

Removing Rows

The result from the converting in the example above gave us a NaT value, which can be handled as a NULL value, and we can remove the row by using the dropna() method.

Ex7: Remove rows with a NULL value in the "155 lb" column:

```
df.dropna(subset=['155 lb'], inplace = True)
```

Removing Rows

Another way of handling wrong data is to remove the rows that contains wrong data.

This way you do not have to find out what to replace them with, and there is a good chance you do not need them to do your analyses.

Ex8:Delete rows where "Duration" is higher than 120:

for x in df.index:

if df.loc[x, "Duration"] > 120:

df.drop(x, inplace = True)

Lab 3: Performing operations on Dataframe

Ex: Creating Dictionary, converting it into data frame and performing operation ssuch as aggregation

#importing pandas module

import pandas as pd

importing numpy as np

import numpy as np

Define a dictionary containing employee data

```
data1 = {'Name': ['Jai', 'Anuj', 'Jai', 'Princi',  
                'Gaurav', 'Anuj', 'Princi', 'Abhi'],
```


PE2-23EE1613 : Data Analytics Lab

```
'Age': [27, 24, 22, 32,
        33, 36, 27, 32],
'Address': ['Nagpur', 'Kanpur', 'Allahabad', 'Kannuaj',
            'Jaunpur', 'Kanpur', 'Allahabad', 'Aligarh'],
'Qualification': ['Msc', 'MA', 'MCA', 'Phd',
                  'B.Tech', 'B.com', 'Msc', 'MA']}]}
```

Convert the dictionary into DataFrame

```
df = pd.DataFrame(data1)
```

performing aggregation using

aggregate method

```
grp1 = df.groupby('Name')
```

```
grp1.aggregate(np.sum)
```

Output:

Age	
Name	
Abhi	32
Anuj	60
Gaurav	33
Jai	49
Princi	59

Use of sum aggregate function on dataset

Python Pandas Concatenate Dataframe Python3

importing pandas module

```
import pandas as pd
```

Define a dictionary containing employee data

```
data1 = {'key': ['K0', 'K1', 'K2', 'K3'],  
  
         'Name': ['Jai', 'Princi', 'Gaurav', 'Anuj'],  
  
         'Age': [27, 24, 22, 32],}
```

Define a dictionary containing employee data

```
data2 = {'key': ['K0', 'K1', 'K2', 'K3'],
```

PE2-23EE1613 : Data Analytics Lab

```
'Address':['Nagpur', 'Kanpur', 'Allahabad', 'Kannuaj'],
```

```
'Qualification':['Btech', 'B.A', 'Bcom', 'B.hons']}]}
```

```
# Convert the dictionary into DataFrame
```

```
df = pd.DataFrame(data1)
```

```
# Convert the dictionary into DataFrame
```

```
df1 = pd.DataFrame(data2)
```

```
display(df, df1)
```

```
# combining series and dataframe
```

```
res = pd.concat([df, df1], axis=1)
```

```
res
```

Output:

	key	Name	Age
0	K0	Jai	27
1	K1	Princi	24
2	K2	Gaurav	22
3	K3	Anuj	32

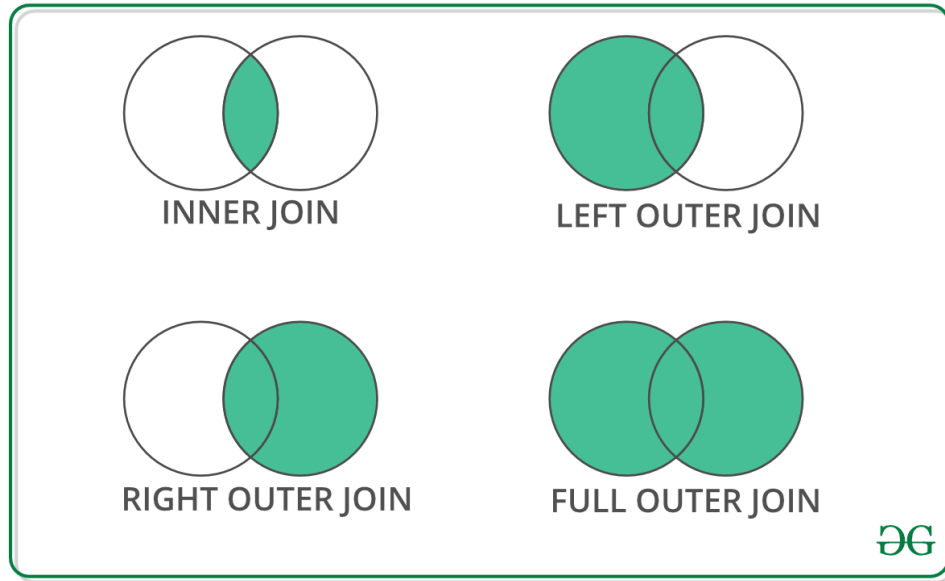
	key	Address	Qualification
0	K0	Nagpur	Btech
1	K1	Kanpur	B.A
2	K2	Allahabad	Bcom
3	K3	Kannauj	B.hons

	key	Name	Age	key	Address	Qualification
0	K0	Jai	27	K0	Nagpur	Btech
1	K1	Princi	24	K1	Kanpur	B.A
2	K2	Gaurav	22	K2	Allahabad	Bcom
3	K3	Anuj	32	K3	Kannauj	B.hons

Merging DataFrame

When we need to combine very large DataFrames, joins serve as a powerful way to perform these operations swiftly. Joins can only be done on two DataFrames at a time, denoted as left and right tables. The key is the common column that the two DataFrames will be joined on. It's a good practice to use keys that have unique values throughout the column to avoid unintended duplication of row values. Pandas provide a single function, [merge\(\)](#), as the entry point for all standard database join operations between DataFrame objects.

There are four basic ways to handle the join (inner, left, right, and outer), depending on which rows must retain their data.



Python Pandas Merge Dataframe Python3

```
# importing pandas module
```

```
import pandas as pd
```

```
# Define a dictionary containing employee data
```

```
data1 = {'key': ['K0', 'K1', 'K2', 'K3'],
```

```
        'Name': ['Jai', 'Princi', 'Gaurav', 'Anuj'],
```

```
        'Age': [27, 24, 22, 32],}
```

```
# Define a dictionary containing employee data
```

PE2-23EE1613 : Data Analytics Lab

```
data2 = {'key': ['K0', 'K1', 'K2', 'K3'],  
         'Address': ['Nagpur', 'Kanpur', 'Allahabad', 'Kannuaj'],  
         'Qualification': ['Btech', 'B.A', 'Bcom', 'B.hons']}
```

```
# Convert the dictionary into DataFrame
```

```
df = pd.DataFrame(data1)
```

```
# Convert the dictionary into DataFrame
```

```
df1 = pd.DataFrame(data2)
```

```
display(df, df1)
```

```
# using .merge() function
```

```
res = pd.merge(df, df1, on='key')
```

res

Output:

	key	Name	Age
0	K0	Jai	27
1	K1	Princi	24
2	K2	Gaurav	22
3	K3	Anuj	32

	key	Address	Qualification
0	K0	Nagpur	Btech
1	K1	Kanpur	B.A
2	K2	Allahabad	Bcom
3	K3	Kannuaj	B.hons

	key	Name	Age	Address	Qualification
0	K0	Jai	27	Nagpur	Btech
1	K1	Princi	24	Kanpur	B.A
2	K2	Gaurav	22	Allahabad	Bcom
3	K3	Anuj	32	Kannuaj	B.hons

Concatinating Two datasets

Joining DataFrame

In order to join the dataframe, we use .join() function this function is used for combining the columns of two potentially differently indexed DataFrames into a single result DataFrame.

Python Pandas Join Dataframe

Python3

```
# importing pandas module
```

PE2-23EE1613 : Data Analytics Lab

```
import pandas as pd
```

```
# Define a dictionary containing employee data
```

```
data1 = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
```

```
        'Age':[27, 24, 22, 32]}
```

```
# Define a dictionary containing employee data
```

```
data2 = {'Address':['Allahabad', 'Kannuaj', 'Allahabad', 'Kannuaj'],
```

```
        'Qualification':['MCA', 'Phd', 'Bcom', 'B.hons']}
```

```
# Convert the dictionary into DataFrame
```

```
df = pd.DataFrame(data1, index=['K0', 'K1', 'K2', 'K3'])
```

```
# Convert the dictionary into DataFrame
```

```
df1 = pd.DataFrame(data2, index=['K0', 'K2', 'K3', 'K4'])
```



```
display(df, df1)
```

```
# joining dataframe
```

```
res = df.join(df1)
```

```
res
```

Output:

	Name	Age
K0	Jai	27
K1	Princi	24
K2	Gaurav	22
K3	Anuj	32

	Address	Qualification
K0	Allahabad	MCA
K2	Kannuaj	Phd
K3	Allahabad	Bcom
K4	Kannuaj	B.hons

	Name	Age	Address	Qualification
K0	Jai	27	Allahabad	MCA
K1	Princi	24	NaN	NaN
K2	Gaurav	22	Kannuaj	Phd
K3	Anuj	32	Allahabad	Bcom

Lab 4: Demonstrate the following data preprocessing tasks using python library a) Dealing with categorical data b) Scaling the features.

Part a)

Aim

To implement Label Encoding and One-Hot Encoding using Python libraries.

Theory

Categorical data must be converted into numerical format for machine learning models.

Label Encoding assigns integers to categories, while One-Hot Encoding creates binary columns.

Procedure with Code

Step 1: Import Libraries

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

Step 2: Create Dataset

```
data = {'Color': ['Red', 'Green', 'Blue', 'Green', 'Red']}
df = pd.DataFrame(data)
print(df)
```

Step 3: Label Encoding

```
le = LabelEncoder()
df['Color_Label'] = le.fit_transform(df['Color'])
print(df)
```

Step 4: One-Hot Encoding

```
df_onehot = pd.get_dummies(df['Color'])
print(df_onehot)
```

Step 5: Combine Data

```
df_final = pd.concat([df, df_onehot], axis=1)
print(df_final)
```

Exercise :

Download Titanic data set from Kaggle and implement Label and One hot encoding on the categorical variables :

Embarked (C, Q, S) , Pclass (1st, 2nd, 3rd)

Part b)

Aim

To perform feature scaling using:

- **Standardization (Z-score normalization)**
- **Min-Max Normalization**

Theory

Feature scaling is a technique used to standardize the range of independent variables in a dataset.

Standardization transforms the data such that it has a mean of 0 and a standard deviation of 1.

Min-Max normalization scales the data to a fixed range, typically between 0 and 1.

Step 1: Import Libraries

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

Step 2: Create Dataset

```
data = {
    'Age': [20, 25, 30, 35, 40],
    'Salary': [20000, 30000, 40000, 50000, 60000]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Step 3: Apply Standardization

```
scaler = StandardScaler()

df_standardized = scaler.fit_transform(df)

print("Standardized Data:")
print(df_standardized)
```

Step 4: Apply Min-Max Normalization

```
minmax = MinMaxScaler()

df_normalized = minmax.fit_transform(df)

print("Normalized Data:")
print(df_normalized)
```

Output:

Age Salary

```
0  20  20000
1  25  30000
2  30  40000
3  35  50000
4  40  60000
```

Standardized Data:

```
[[-1.41421356 -1.41421356]
 [-0.70710678 -0.70710678]
 [ 0.         0.        ]
 [ 0.70710678  0.70710678]
 [ 1.41421356  1.41421356]]
```

PE2-23EE1613 : Data Analytics Lab

Normalized Data:

[[0. 0.]

[0.25 0.25]

[0.5 0.5]

[0.75 0.75]

[1. 1.]]

Lab 5a: Data visualization using Matplotlib

Python offers a wide range of libraries for data visualization, with Matplotlib, Seaborn, Plotly, and Bokeh being some of the most popular ones. Here are some data visualization examples using Python:

Import Matplotlib:

First, you need to import Matplotlib. You can use the pyplot module for most basic plotting tasks:

```
import matplotlib.pyplot as plt
```

1. Line Chart with Matplotlib:

```
import matplotlib.pyplot as plt
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a plot
```

```
plt.plot(x, y, label='Linear Function')
```

```
plt.xlabel('X-axis')
```

```
plt.ylabel('Y-axis')
```

```
plt.title('Simple Line Plot')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
# Display the plot
```

```
plt.show()
```

2. Scatter Plot: Scatter plots are useful for visualizing the distribution of individual data points.

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 12, 5, 7, 9]
```

```
plt.scatter(x, y, label='Data Points', color='blue', marker='o')
```

```
plt.xlabel('X-axis')
```

```
plt.ylabel('Y-axis')
```

```
plt.title('Scatter Plot Example')  
plt.legend()  
plt.show()
```

3. Bar Chart: Bar charts are effective for comparing data between different categories.

```
import matplotlib.pyplot as plt  
  
categories = ['A', 'B', 'C', 'D']  
values = [10, 15, 7, 12]  
plt.bar(categories, values)  
plt.xlabel('Categories')  
plt.ylabel('Values')  
plt.title('Bar Chart Example')  
plt.show()
```

4. Histogram: Histograms help visualize the distribution of a single variable.

```
import matplotlib.pyplot as plt  
import numpy as np  
  
data = np.random.randn(1000) # Generate random data  
plt.hist(data, bins=20, color='skyblue')  
plt.xlabel('Value')  
plt.ylabel('Frequency')  
plt.title('Histogram Example')  
plt.show()
```

5. Pie Chart: Pie charts are suitable for showing parts of a whole.

```
import matplotlib.pyplot as plt  
  
labels = ['A', 'B', 'C', 'D']  
sizes = [15, 30, 45, 10]  
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)  
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.  
plt.title('Pie Chart Example')
```

```
plt.show()
```

6. Box Plot: Box plots display the distribution and skewness of a dataset.

```
import matplotlib.pyplot as plt
import numpy as np
data = np.random.randn(100)
plt.boxplot(data)
plt.xlabel('Data')
plt.ylabel('Values')
plt.title('Box Plot Example')
plt.show()
```

Exercise : Write a Python Program to Get Total Price of all FuelType from Toyota.csv file and show it using a line plot with the following Style properties. Generated line plot must include following Style properties

- Line Style dotted and Line-color should be red
- Show legend at the lower right location.
- X label name = Fuel Type
- Y label name = Price

Lab 5b: Data visualization using Seaborn

Lab Tasks:

Task 1: Import Required Libraries

```
import seaborn as sns  
  
import matplotlib.pyplot as plt  
  
import pandas as pd
```

Task 2: Load Sample Data

```
# Load the 'tips' dataset from Seaborn  
  
tips_data = sns.load_dataset('tips')  
  
...
```

Task 3: Data Exploration

```
# Display the first few rows of the dataset  
  
print(tips_data.head())  
  
# Get summary statistics of the dataset  
  
print(tips_data.describe())
```

Task 4: Data Visualization

1. Create a Histogram

```
# Create a histogram of total bill amounts  
  
plt.figure(figsize=(8, 6))  
  
sns.histplot(tips_data['total_bill'], bins=15, kde=True, color='skyblue')  
  
plt.title('Distribution of Total Bill Amounts')  
  
plt.xlabel('Total Bill Amount ($)')  
  
plt.ylabel('Frequency')  
  
plt.show()
```

2. Create a Scatter Plot

```
# Create a scatter plot of total bill vs tip

plt.figure(figsize=(8, 6))

sns.scatterplot(x='total_bill', y='tip', data=tips_data, color='coral')

plt.title('Total Bill Amount vs Tip')

plt.xlabel('Total Bill Amount ($)')

plt.ylabel('Tip Amount ($)')

plt.show()

'''
```

3. Create a Box Plot

```
# Create a box plot of total bill amounts by day

plt.figure(figsize=(8, 6))

sns.boxplot(x='day', y='total_bill', data=tips_data, palette='coolwarm')

plt.title('Total Bill Amounts by Day')

plt.xlabel('Day of the Week')

plt.ylabel('Total Bill Amount ($)')

plt.show()
```

4. Create a Heatmap

```
# Create a correlation heatmap for the numerical columns

correlation_matrix = tips_data.corr()

plt.figure(figsize=(8, 6))

sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')

plt.title('Correlation Heatmap')

plt.show()
```

Task 5: Additional Customizations

Experiment with various Seaborn functions and customize your visualizations further. You can explore features like hue, style, col, row parameters to add more dimensions to your plots.

Task 6: Save and Share Your Visualizations

Save your visualizations as image files

```
plt.figure(figsize=(8, 6))
```

```
sns.barplot(x='day', y='total_bill', data=tips_data, estimator=sum, ci=None, palette='pastel')
```

```
plt.title('Total Bill Amounts by Day')
```

```
plt.xlabel('Day of the Week')
```

```
plt.ylabel('Total Bill Amount ($)')
```

```
plt.savefig('total_bill_by_day.png')
```

Lab 6 : Perform Ztest and T test on data for hypothesis testing

Aim: To implement the T-test model

Steps:

1. Import the required packages
2. Create the two variable x & y
3. Assign the values to the 2 variable
4. Find the standardization of it
5. Print the result
6. End the result

Program:

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats
np.random.seed(42)
scores_uni_A = np.random.normal(loc=80, scale=8, size=30) # Mean=80, SD=8, N=30
scores_uni_B = np.random.normal(loc=75, scale=10, size=30) # Mean=75, SD=10, N=30
t_stat, t_p_value = stats.ttest_ind(scores_uni_A, scores_uni_B)
mean_diff = np.mean(scores_uni_A) - np.mean(scores_uni_B)
pooled_std = np.sqrt((8**2 / len(scores_uni_A)) + (10**2 / len(scores_uni_B)))
z_stat = mean_diff / pooled_std
z_p_value = 2 * (1 - stats.norm.cdf(np.abs(z_stat))) # Two-tailed z-test
print(f"T-test: t-stat = {t_stat:.3f}, p-value = {t_p_value:.3f}")
print(f"Z-test: z-stat = {z_stat:.3f}, p-value = {z_p_value:.3f}")
import warnings
warnings.filterwarnings('ignore')

sns.histplot(scores_uni_A, kde=True, color='blue', label='University A', stat='density')
sns.histplot(scores_uni_B, kde=True, color='green', label='University B', stat='density')
plt.title('Technical Assessment Scores: University A vs University B')
plt.legend()
```

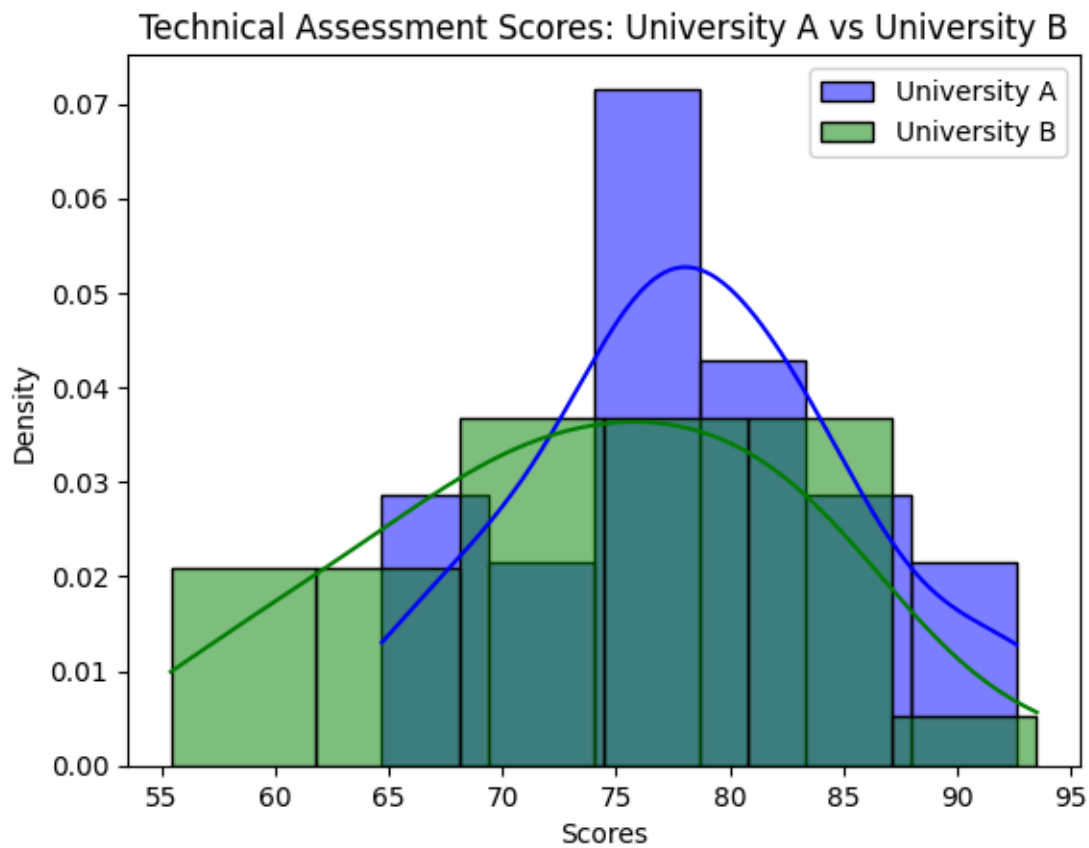
```
plt.xlabel('Scores')
```

```
plt.ylabel('Density')
```

```
plt.show()
```

T-test: t-stat = 2.190, p-value = 0.033

Z-test: z-stat = 2.013, p-value = 0.044

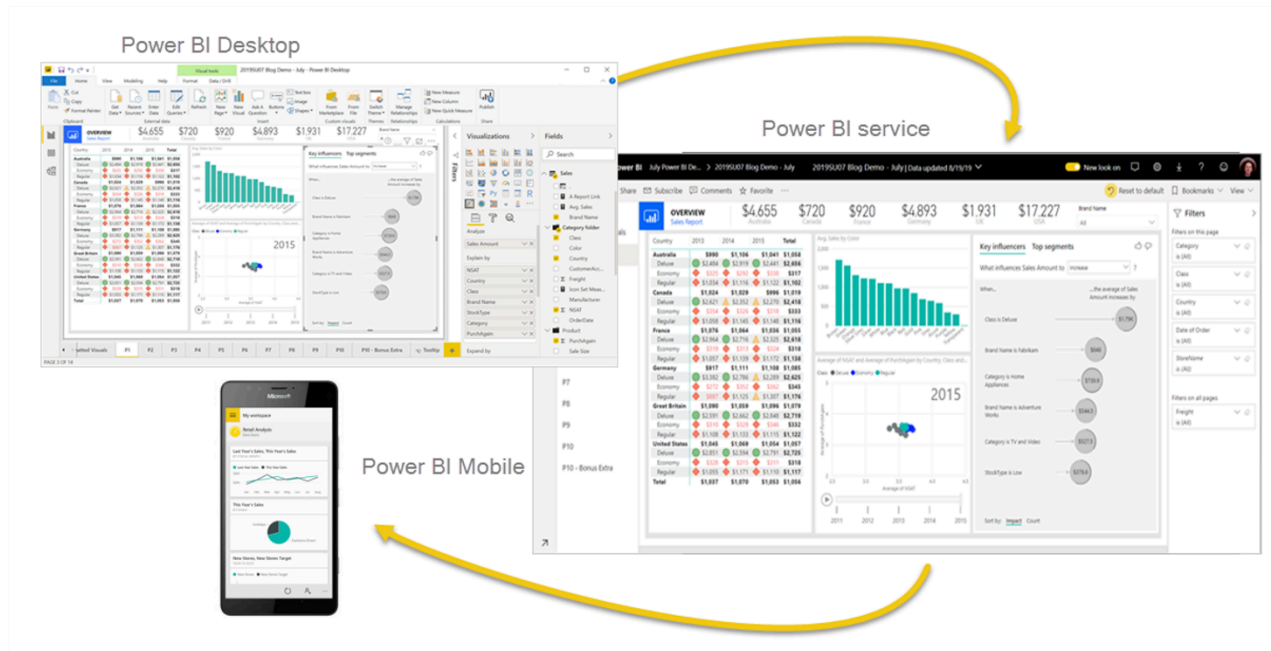


Lab 7 : To implement the program of Anova using the python program

From customer and employee data metrics for company goals, to sales and acquisitions, businesses are drowning in data, but this data is only as good as your ability to interpret and communicate its meaning. That's where Power BI (Business Intelligence) comes into play.



PE2-23EE1613 : Data Analytics Lab



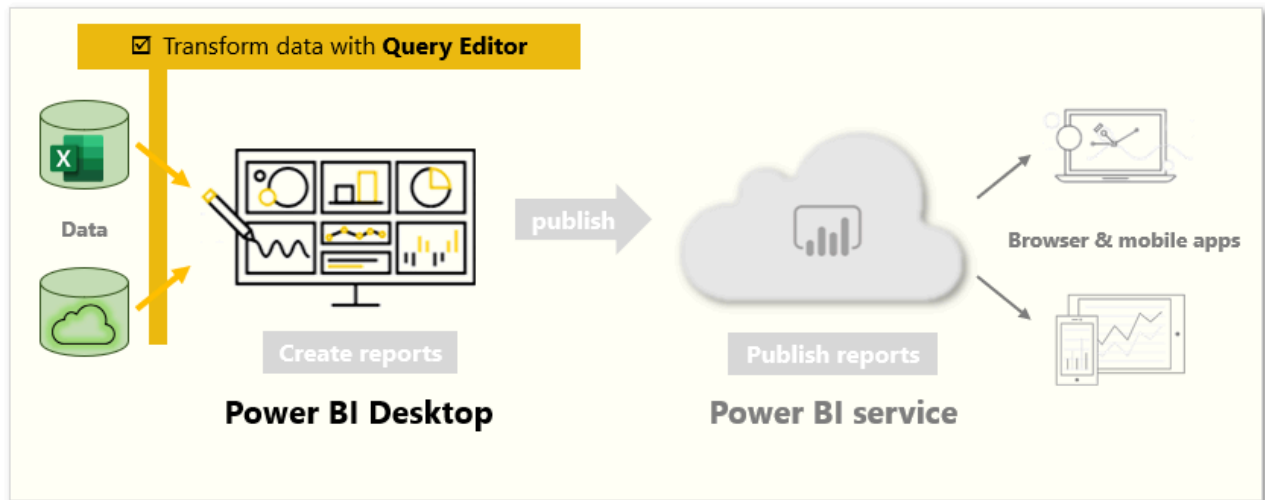
These three elements, Power BI Desktop, Power BI Service, and Power BI Mobile apps are designed to let people create, share, and consume business insights in the way that serves them, or their role, most effectively.

When working with data from different data sources, it's not always going to be in a format that allows it to be displayed with other data. For example, you might be getting ready to build a device usage report that display IoT related device details for individual customers. Typically, that data exists in multiple systems. The customer details likely are in your organization's Customer Relationship Management (CRM) system, while the IoT data is likely stored in a dedicated IoT system such as Azure IoT Hub. Many times, the IoT data isn't structured as nicely as the data in your CRM system. Some of the data may not be in the right format, or there might simply be more data than you need. In these instances, you'll need to clean and transform your data. Cleaning and transforming data is how you prepare your data and get it ready to be used. To begin transforming and cleaning data, you use the Power BI Desktop application.

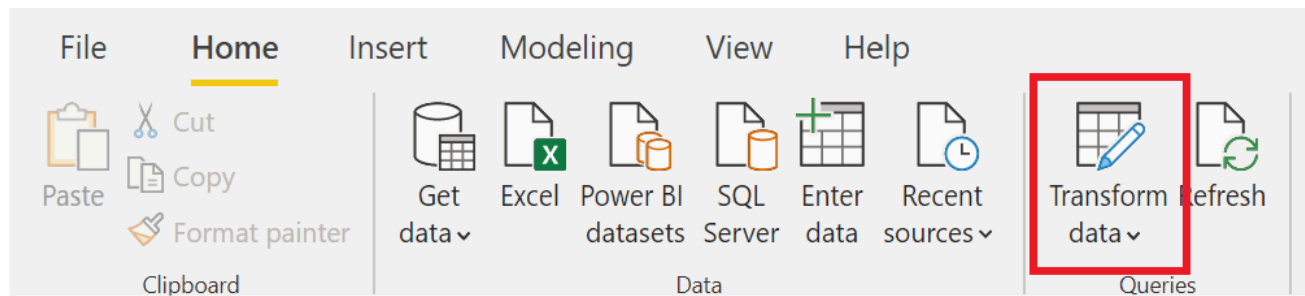
Power BI Desktop has three views:

- **Report view:** You can create queries to build compelling visualizations that you can share with others. You can arrange them as you want them to appear.
- **Data view:** See the data in your report in data model format, where you can add measures, create new columns, and manage relationships.
- **Model view:** Get a graphical representation of the relationships that are established in your data model and manage or modify them as needed.

Power BI Desktop includes the **Power Query Editor** tool, which can help you shape and transform data so that it's ready for your models and visualizations.



To begin, select **Edit** from the **Navigator** window to launch Power Query Editor. You can also launch Power Query Editor directly from Power BI Desktop by using the **Transform Data** button on the Home ribbon.



Transforming data

As mentioned previously, transforming data is the process of putting data into a format that is useable in your reports. Examples of the available transformations include removing a column from the table, duplicating the column under a new name, or replacing values.

PE2-23EE1613 : Data Analytics Lab

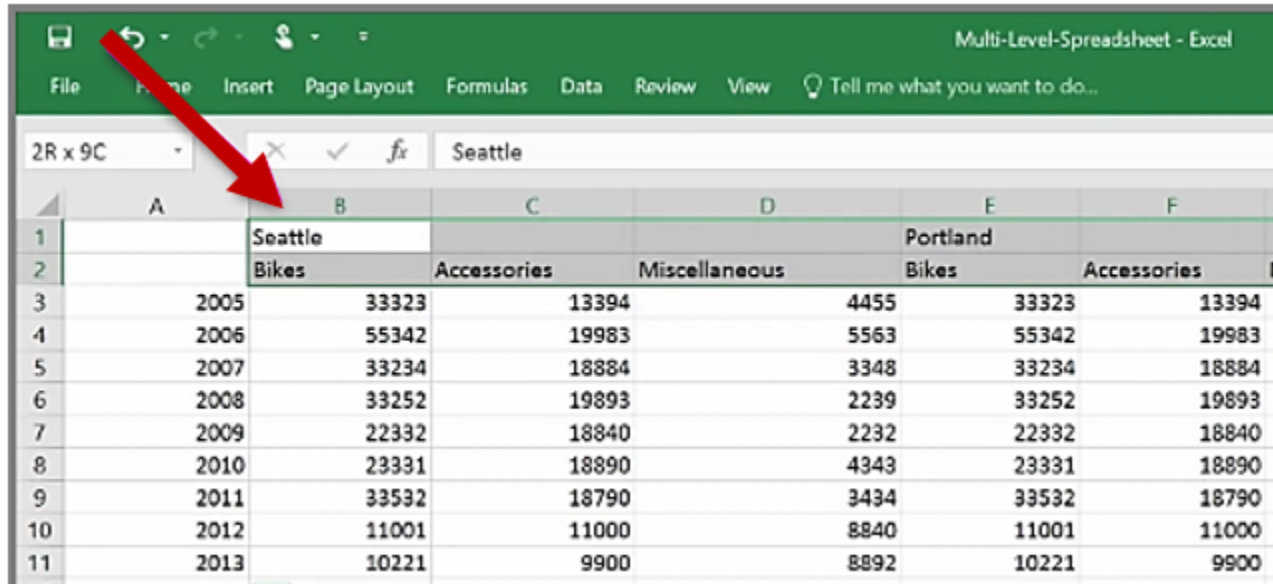
1 ² ₃ Crime	1 ² ₃ Culture	1 ² ₃ Health care quality
Copy	10	12
Remove	15	10
Remove Other Columns	31	8
Duplicate Column	9	5
Add Column From Examples...	26	36
Remove Duplicates	7	19
Remove Errors	40	30
	16	22
Change Type	Decimal Number	12
Transform	Fixed decimal number	26
1 ₂ Replace Values...	☒ Whole Number	17
Replace Errors...	Percentage	3
Group By...	Date/Time	24
Fill	Date	32
Unpivot Columns	Time	35
Unpivot Other Columns	Date/Time/Timezone	12
Unpivot Only Selected Columns	Duration	15
		44

Clean data

While Power BI can import your data from almost any source, its visualization and modeling tools work best with columnar data. Sometimes, your data isn't formatted in simple columns, which is often the case with Excel spreadsheets.

A table layout that looks good to the human eye might not be optimal for automated queries. For example, the following spreadsheet has headers that span multiple columns.

PE2-23EE1613 : Data Analytics Lab



	A	B	C	D	E	F
1		Seattle			Portland	
2		Bikes	Accessories	Miscellaneous	Bikes	Accessories
3	2005	33323	13394	4455	33323	13394
4	2006	55342	19983	5563	55342	19983
5	2007	33234	18884	3348	33234	18884
6	2008	33252	19893	2239	33252	19893
7	2009	22332	18840	2232	22332	18840
8	2010	23331	18890	4343	23331	18890
9	2011	33532	18790	3434	33532	18790
10	2012	11001	11000	8840	11001	11000
11	2013	10221	9900	8892	10221	9900

When you clean data, you might combine those rows into a single item to better format the data to fit your needs. Or you may have a series of numeric data that would need to be aggregated to display better. With Power Query, there are a series of tools that you can use to prepare the data.

Lab 8 : Building dashboard in Power BI

Build a basic dashboard

You need to take the following steps:

- **Prepare your data:** Preparing the data ensures that it's in a format that Power BI can easily consume.
- **Build a report:** The report contains the visuals that you want to include in your dashboard. Depending on the scenario, reports can be built in either Power BI Desktop or using the Power BI Service.
- **Pin the report visuals to a dashboard:** Dashboards are the primary element that users use for viewing data. They can include data from multiple reports as needed.
- **Share a link to the dashboard:** Any users with the link and the necessary permissions are easily able to view and interact with the data.

Prepare the data

The first thing you need to do is ensure that your data is ready to be consumed. Depending on the data source and the volume of data you're working with, this might include doing some data cleansing and transforming using Power Query. In the case of connecting to an Excel spreadsheet, we want to make sure the data is in a flat table, and that each column contains the right data type. For example, text, date, number, or currency. It's also important that you have a header row but no columns or rows that display totals. Total operations will be handled in Power BI as we create the visuals.

E	F	G	H
Product	Date	Units Sold	Manufacturing f
Carretera	1/1/2014	1618.5	\$ 3.00
Carretera	1/1/2014	1321	\$ 3.00
Carretera	6/1/2014	2178	\$ 3.00
Carretera	6/1/2014	888	\$ 3.00
Carretera	6/1/2014	2470	\$ 3.00
Carretera	12/1/2014	1513	\$ 3.00

Diagram illustrating data types associated with columns:

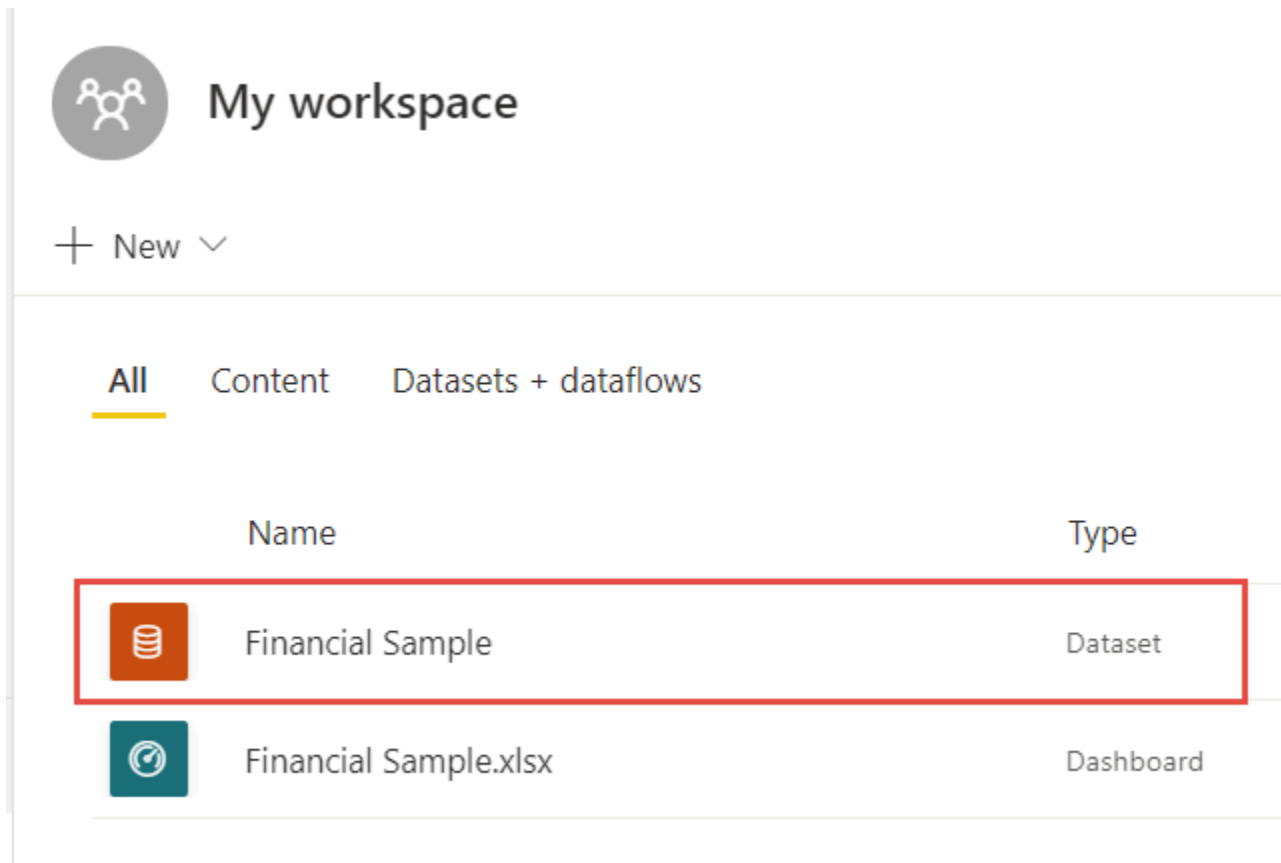
- Product (Text)
- Date (Date)
- Units Sold (Number)
- Manufacturing f (Currency)

You can see that there's a header column, and each column has the correct data type associated with the data.

Upload your data to the Power BI service

The Power BI service is where you'll be able to create reports that connect to your data sources. This includes Excel files that live on your computer. With a few simple clicks, you can attach to a dataset, and Power BI will create a blank dashboard where you'll be able to place visuals later.

In the image, we attached to sample financial data. It shows the completed Financial Sample dataset, as well as the blank dashboard.

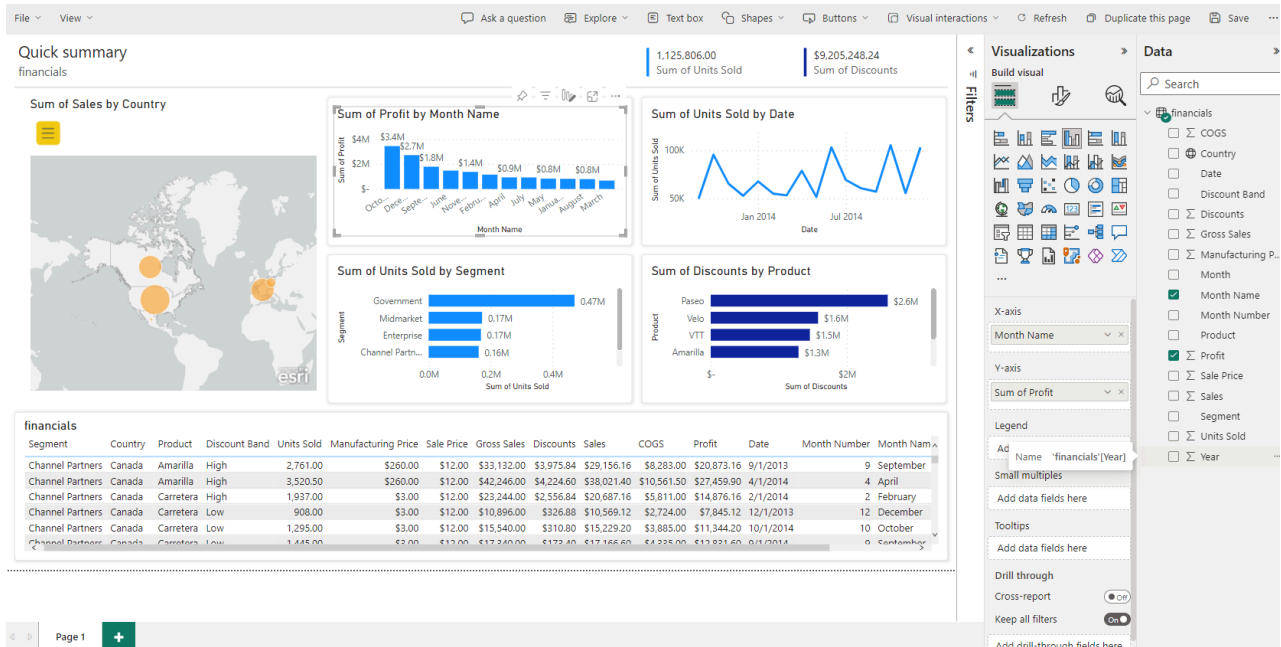


Build your report

Once you connected to your data, you can either create a new report or edit an existing report that was created previously. Then we can begin editing the report by using the editing view. On the right are the **Visualizations**, **Filters**, and **Fields** panes. Your Excel workbook table data appears in the **Fields** pane. At the top is the name of the table, **financials**. Under that, Power BI lists the column headings as individual fields.

Different visualizations are available that you can use to display the data in your report. Different visualizations can be added to the report based on the data that you want to communicate. In the image, multiple bar charts are being used to showcase the number of units being sold. Additionally, map control is included to show sales volumes per country/region.

PE2-23EE1613 : Data Analytics Lab



Each visualization includes a series of filters and controls that can be used to impact how the data is being presented. For example, if you want to change the sum of units sold to display profit by date, you could switch the Y-axis from **Sum of Units sold**, to **Sum of Profit**.

More and more visualizations are being added to Power BI to reflect how business is being done. For example, there's a Power Apps visualization that lets you build a canvas app directly in your Power BI report, which is connected to the Power BI data set. As people interact with the data, the data in the Power App changes. For example, you could create a canvas app that includes actions such as sending emails or scheduling meetings. As you drill down on a Power BI report, you identify customers we have not talked to in a while. The embedded canvas app lets us initiate actions, such as sending an email, right from the Power BI report.

Pin to a dashboard

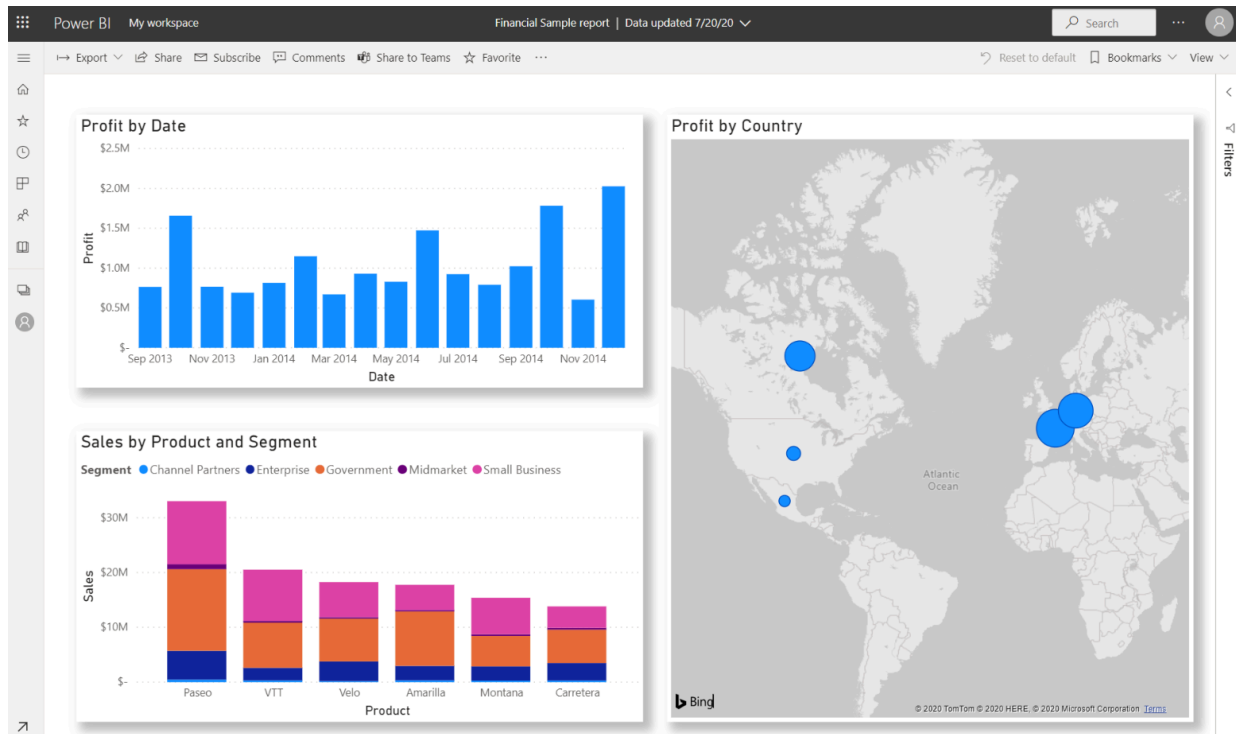
After you have all your visualizations on your reports, you can build your dashboard. Dashboards are easy to build because you are just determining which visuals from your created reports you want to include.

In the image, we are taking the Profit by Date visualization and pinning it to a dashboard.

PE2-23EE1613 : Data Analytics Lab

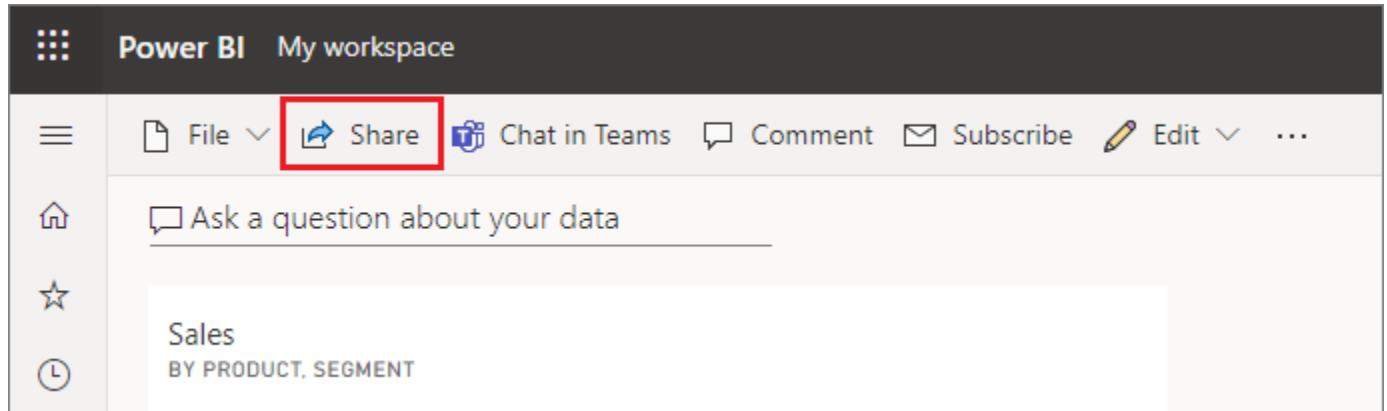


Since dashboards can have visuals from multiple reports, it makes it easy to have very detailed dashboards that include data from multiple data sources, even if some of those data sources might be unrelated.



Share a link to your dashboard

Initially dashboards that you create will only be visible to the person that created them. However, Power BI makes it simple to share dashboards with other individuals. You can share your dashboard and underlying report with any colleague who has a Power BI account. They'll be able to interact with it, but they cannot save changes. If you allow it, they can reshare with others or build a new report based on the underlying dataset.



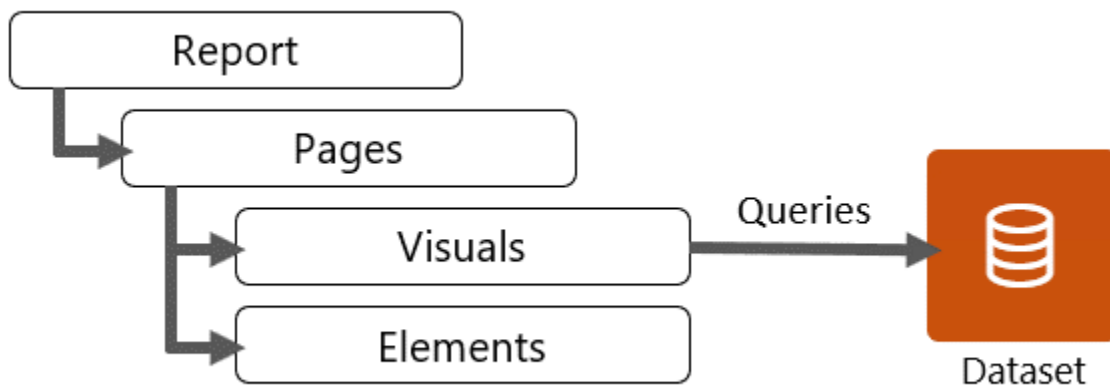
Lab 9 : Design Power BI reports

You can design Microsoft Power BI reports in Power BI Desktop or Power BI service (web portal). Power BI mobile apps don't support report design; they support only the report consumer experience.

Report structure

Structurally, a Power BI report connects to a single dataset (data model), and it has at least one report page. However, it's common that reports have multiple pages. On each page, *report objects* are laid out. Report objects include:

- **Visuals** - Visualizations of dataset data.
- **Elements** - Provide visual interest but don't use dataset data. Elements include text boxes, buttons, shapes, and images.
- Similar to Microsoft Excel worksheets, you can add, rename, resequence, hide, duplicate, or delete Power BI report pages.



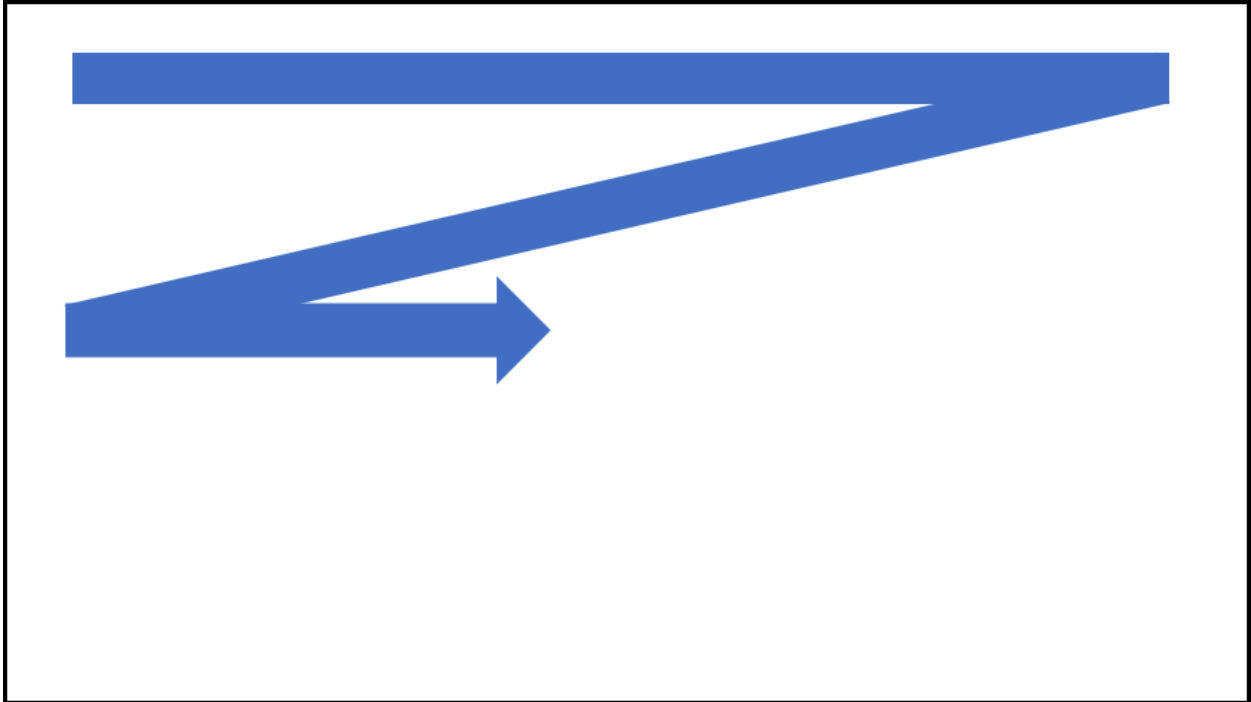
Design the analytical report layout

An analytical report can consist of one or more pages, and pages comprise report objects. Report objects can be data visuals, which represent query results, or decorations like images, background shapes, or text. You can begin to design a report layout by determining the number, sequence, and purpose of the pages. Make sure that you avoid combining subjects or opposing objectives on the same page. Then, design each page layout with specific report objects that are relevant to the requirements.

Additionally, good report design layouts should consider design principles of placement, balance, contrast, proximity, and repetition.

Placement

Good placement of report objects contributes to an ordered report design. Generally, you should place the most important information in the upper-left corner of the page and arrange the report elements from left to right and top to bottom.



Arrange report objects so that the vertical and horizontal edges align because it looks ordered and is pleasing to the eye. Position-related report objects in logical groups. An ordered report layout creates a connection between visual elements and avoids clutter that can result from a seemingly random placement of report objects.

Additionally, aligning report objects in a visually pleasing layout can convey more energy and interest than simply centering or randomly placing report objects. Consider applying the *rule of thirds*, which is a visual arts rule that can be applied to report object placements in an analytical report. The rule proposes that a page layout should be divided into an invisible grid of nine equal parts. The grid is formed by two equally spaced horizontal lines and two equally spaced vertical lines. Then, report objects can be placed within the cells of the grid.

Complete self paced course module from power BI desktop on following topics as submission for lab 7 , lab 8 and lab 9.

PE2-23EE1613 : Data Analytics Lab

- 1. Get started with power BI**
- 2. Get and transform data in Power BI**
- 3. Explore fundamentals of data visualization**